

A FIPA-Compliant Framework for Integrating Rule Engines into Software Agents for Supporting Communication and Collaboration in a Multiagent Platform

Francisco J. Aguayo-Canela¹, Héctor Alaiz-Moretón²,
Isaías García-Rodríguez²(✉), Carmen Benavides-Cuéllar²,
José Alberto Benítez-Andrades², and Paulo Novais³

¹ IEEE Power & Energy Society, Seville, Spain
francisco.aguayo@ieee.org

² Department of Electrical and Systems Engineering and Automation,
Universidad de León, León, Spain
{hector.moreton, isaias.garcia,
carmen.benavides, jbenas}@unileon.es

³ Algoritmi Centre/Department of Informatics,
University of Minho, Braga, Portugal
pjon@di.uminho.pt

Abstract. Production rules have been traditionally considered a good knowledge representation formalism for creating expert systems, and also as a good mechanism for building intelligence within software agents due to the flexibility of their declarative knowledge representation. But the use of rules and rule engines for implementing behaviors inside agents in the context of a multiagent environment has a number of challenges if one wants to adhere and preserve the defining characteristics of agency and support the kind of communication and collaboration mechanisms needed by multiagent systems. This paper describes a framework, and its implementation, for the integration of production rules reasoning mechanisms inside software agents in the context of a multiagent platform, preserving the defining characteristics of agency and accomplishing the well-established FIPA standards for agent interaction.

The implementation has been accomplished by using either CLIPS or Jess as the rule engine, but it can be extended to other formalisms. JADE was chosen as the multiagent platform for developing and testing the solution. The proposed framework includes the uncoupled integration of the rule inference engine into the agent and the adherence to FIPA specifications about using protocols and communication processes. The resulting framework can be used to adapt legacy expert systems whose functionality can be divided to be performed by separate agents or to build new, distributed intelligent systems exploiting the capacities of a multiagent platform at different levels.

To show the validity of the approach, a functional test environment was built and is available for downloading.

Keywords: Rule-based agent · Multiagent systems · JADE · CLIPS · Jess · FIPA · Interaction protocol

1 Introduction

Production rules is a well-known knowledge representation formalism that has been extensively used for building intelligent systems. The spread of microcomputers and their increasing computation power made it possible to study and build parallel and distributed environments for running intelligent applications much faster and with greater flexibility and modularity than with traditional, monolithic rule-based expert systems [1, 2].

With the research on the agent abstraction and the multiagent distributed system paradigm, the rule-based systems found another field of application. Rules, as the building blocks for performing the agent behavior, as well as coordination paradigms for letting the rule-based agents collaborate towards a common objective, resulted in the so-called multiagent production systems, which are conceptually different from parallel and distributed production systems. While in the case of distributed production systems the aim is to distribute a monolithic expert system in order to run it faster, in the case of the multiagent paradigm the objective is to make different, independent production systems, work together with a common goal by coordinating their activities [3].

The first step towards building a multiagent production system is the integration of a rule engine into a software agent. Then, a coordination mechanism and a collaboration architecture are also needed in order to obtain the collaborative nature that multiagent systems require. The integration of the rule engine into the agent is crucial to achieve the coordination of the agents and also the flexibility needed to build a multiagent system of this nature, this integration must be accomplished by having into account the future coordination and communication mechanisms that may be needed.

The objectives of the present work regarding the integration of a rule engine into an agent are:

- The integration must be neutral with respect to the particular technology of the rule engine (no assumption or knowledge is needed for one agent to communicate with another one regarding the particular rule engine that is running in the receiving agent).
- Agents with a rule-based behavior must be able to collaborate in a number of ways with other rule-based agents, both by direct communication between them and also by collaboration using a shared knowledge repository.
- Actions performed in the rule-based system of a given agent must be fully and exclusively controlled by the agent itself, separately from other activities or behaviors.
- The rule engine must not block the agent basal behavior (for example, the ability to continue receiving ACL messages) while performing the reasoning.
- The design must help and ease the development of rule-based multiagent applications.

The platform chosen for building the practical solution is JADE [4]. This election is based on its maturity, the size of the user community, and the use of behavior-based agents, which are susceptible to allow the integration of rule engines as their behaviors.

The rest of the paper is organized as follows. Section 2 introduces previous research aimed at the integration of rule engines and rule-based reasoning services into the agents. Section 3 shows the proposed solution, giving a functional view of the different solutions designed for the framework. In Sect. 4, a test environment built for testing and demonstration purposes is presented. This environment is fully functional can be downloaded from the specified URL. Finally, Sect. 5 gives some discussion and points to future research work.

2 Building Agents that Use Rule-Based Reasoning

Rule-based agents allow the use of declarative statements for defining the agent reasoning capabilities. This is a desirable characteristic when the problem-solving technique is not well defined and/or may need a number of adjustments, even at run time. With rule-based agents, a number of reasoning models can be built, from reactive to deliberative ones, as well as hybrid ones [3].

A distinction is usually made between rule-based agents and rule-enhanced agents. A rule-based agent is an agent whose behavior and/or knowledge is totally expressed using rules, while a rule-enhanced agent is the one that use a rule engine as an additional component for specific reasoning capabilities. In any case, the integration strategy can be the same when using a behavior-based agent as the ones used in the JADE platform [5, 6]. In the rest of the document, the agents with an integrated rule engine will be referred as “rule-based agent”.

2.1 Previous Solutions

The approach designed by Cardoso [7] consists in the instantiation of a Jess (Java Expert System Shell) rule engine [8] object within a JADE (one shot or cyclic) behavior. It is one of the first attempts to build the agent reasoning by means of a rule engine. As pointed in [9] this approach has a number of limits:

- The rule engine is completely hidden to other agents of the system, and there is not any support for the cooperation among different rule-based agents.
- The solution is bound to a particular technology: The Java Expert System Shell (Jess) in this case.
- Implementing some of the agent capabilities into the rule engine, as the solution does, is cumbersome (for example parsing ACL messages in Jess).
- The agent may get blocked when the rule engine is running, and then a limit in the number of run cycles of the rule engine must be imposed.

JAMOCHAAGENT [10] introduces a transformation component between the agent and the rule engine that is in charge of transforming FIPA ACL messages (transporting a description of an action to be performed by the rule engine of the receiving agent) into convenient actions for the rule engine. In this case, the CLIPS system [11] is the one used for acting as the rule engine. This functionality achieves a certain degree of flexibility into the system because agents can be re-programmed when the system is running without the necessity of restarting the whole platform. The transformation

between ACL messages and CLIPS statements are based upon direct correspondences between ACL speech acts (Agree, Failure, Inform, etc.) and CLIPS engine actions that are evaluated by production rules inside the rule engine.

Having in mind the limitations found for the Jess-JADE integration [7], a research was conducted that proposed the integration of an open source Java-based rule engine, Drools, into JADE (Drools4JADE, or D4J) [5, 6]. This work also performs the integration of the rule engine as an agent behavior. The work stresses the need of using the agent communication acts as a way of interchanging problem-solving knowledge, that is, rules that may have been developed by an agent but may be useful for other/s. This “rule mobility” also raises security concerns that should be faced in a real scenario. The use of Drools, based on the JAVA language, allows referring to ACL messages both in the precondition and in the consequent parts of the rules. This resulted in the design decision of allowing manipulation of facts and rules on Drools agents through ACL messages by exposing an API for invoking these actions. The integration of a Bean-Shell [12] component into the agent allows manipulating the Drools elements by using JAVA as a scripting language.

More recent approaches use the service-oriented architecture to build a reasoning service for the agents in a multiagent platform. These approaches try to solve the interoperability problem when using different reasoning paradigms and representation languages. EMERALD [13] is a framework for interoperation of intelligent agents in the Semantic Web. The framework integrates different reasoning mechanism into a multiagent platform using a Reasoner agent that, actually, acts more like a web service. EMERALD is based on FIPA specifications, achieving a fully FIPA-compliant model and deals with trust issues [13]. The approach in Rule Responder [14] also focuses on the interoperability of different reasoning paradigms, integrating them in the multiagent platform through a service-oriented architecture; but the solution is not fully compliant with FIPA specifications.

3 The Proposed Solution

Based on the requirements stated in Sect. 1 and the previous research presented in Sect. 2, the final integration solution was designed and implemented. The framework proposed in this work uses an integration of the agent with its rule engine that is both uncoupled, in the sense of being independent of the engine technology, but also having a strong link in another sense because a given engine will be exclusively devoted to the agent where it is instantiated. The building blocks of the solution are presented next, with some details of their implementation.

3.1 Building an Interface for Managing the Rule Engine from the Agent

An agent integrates a rule engine by means of a programming interface (called *RBEngine*) that allows this agent to transparently make use of any rule engine technology by building an implementation for one of the possible technologies to be used.

RBEngine objects are created inside the initialization method for the agent. This way, the agent has control over the functions and responses from the inference engine

and, also, it is possible to delay, up to the instant of agent creation in the platform, the decision about which kind of inference engine technology is to be used for a given agent.

The object representing the rule engine is unique and is subordinated to the object representing the agent and to its execution thread. The rule engine object is created in the moment of the incorporation of the agent to the platform and not as part of the lifecycle of a behavior of the agent.

3.2 Communication Mechanism Between Agents

The mechanism for communication between rule-based agents adheres to FIPA specifications, using ACL messages and a domain ontology for representing the specific meanings of the message contents.

The set of actions that a rule-based agent can ask other to perform on its rule engine is limited to predefined list that includes the typical activities of these types of systems (loading new facts and rules, execute a number of cycles, query some fact values, perform a clear or reset operation, etc.). These actions are designed in accordance to the interface defined for interacting with the rule engine and constitutes both an efficient and secure way of conceptualizing the set of possible operations to be performed in any rule engine that may be used.

3.3 Communication of the Agent with Its Rule Engine

As was previously stated, the actions to be performed by the rule engine are requested by implementing and invoking the *RBEngine* interface from the agent but, before this, the agent has performed another important task.

Figure 1 shows a schema of the communication mechanism described here. When an agent with a rule engine receives an ACL message with actions to be performed in its engine, the agent first captures the message (first loop in Fig. 1), communicates to the sender the acceptance of the action (in the case it is accepted) and, then, builds and sends itself a new ACL message with the same action than the sender proposed (if accepted) and the identification of the conversation and the sender, see second loop in Fig. 1. The sender has, then, a notification of the acceptance of the required actions and, as the rule engine may take some time to obtain the results, the actions to be achieved are placed again in the receiving agent's message queue in order to be finally processed by the rule engine when it is ready to do so.

This way, the agent is free to continue interacting with other agents in the platform. Once the rule engine process and executes the actions, the results of the execution are obtained, and the sender is informed of the results. This mechanism, based on a double implementation of a FIPA interaction protocol (sender agent to receiving agent and receiving agent to itself – to its rule engine –), allows an ordered, controlled, and privately managed way of using the rule engine of the agent. Using a FIPA interaction protocol is a natural way of implementing communication, both between agents and between an agent and its rule engine.

This behavior of the agent is functionally similar to a proxy: the agent analyzes the request, an acceptance message is sent to the sender, and the request, along with an

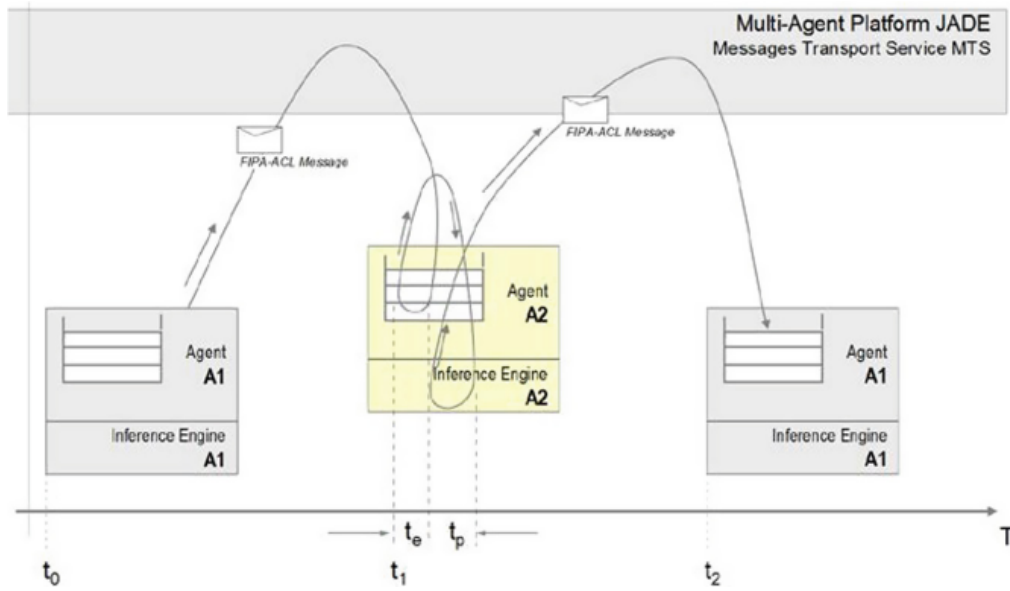


Fig. 1. Schematic view of an agent A1 sending an ACL message targeted at the rule engine in agent A2.

identification of the conversation that originated it, is stored for later processing when the rule engine is ready.

3.4 Execution of the Rule Engine Activities in a Threaded Behavior

When the rule engine is ready to process a new message, a Finite-State-Machine (FSM) behavior is responsible for implementing this processing. The first state of the FSM is a listening behavior, which captures the messages that the agent had previously built and put in its message queue. The second state is an execution behavior that obtains the contents of the message and inserts them into the rule engine. A threaded-behavior wraps this behavior in order to ensure that the execution thread of the agent is not interrupted by the activity of the rule engine. When this activity ends, and the thread of the execution behavior is dead, the third state, the response behavior starts. This behavior captures the output of the engine shell, builds the response message and sends it to the original requester agent by using the original conversation IDs. Finally, the response behavior returns control of the FSM to the listening behavior and the next ACL message for the rule engine in the agent's message queue, if any, is processed.

4 Test Environment

To test the validity of the approach, the integration solution was implemented in the JADE multiagent platform. CLIPS (CLanguage Integrated Production System) and Jess (Java Expert System Shell) were chosen as the rule engines to be integrated due to their popularity, stability and different developing languages (C and Java respectively). Agents with rule engines and the rest of the components were programmed as

previously described. These agents were augmented with a graphical user interface where different windows allowing interactions with the agents were programmed:

- A BeanShell component was included in the agent, including a command window, to manage the internals of the agent by using Java as a scripting language, even at runtime. Figure 2 shows a screenshot.
- A synchronous shell was developed to let a human user interact with the rule engine of any rule-based agent in the platform. The user can interact directly with the rule engine by introducing the specific rule language commands. The window gets blocked until a response is received from the remote engine. This functionality is used for programming, testing and debugging. Figure 3a shows a screenshot of this window.
- An asynchronous shell used for interacting with any rule-based agent using the limited set of commands mentioned in Sect. 3.2 and the non-blocking functionality introduced in Sects. 3.3 and 3.4. This graphical interface allows the human to build the same kind of messages that the agents would build when they are in production. The window is not blocked, and the user can continue sending commands. The responses arrive when the remote engine finishes processing. Figure 3b shows a screenshot of this window.

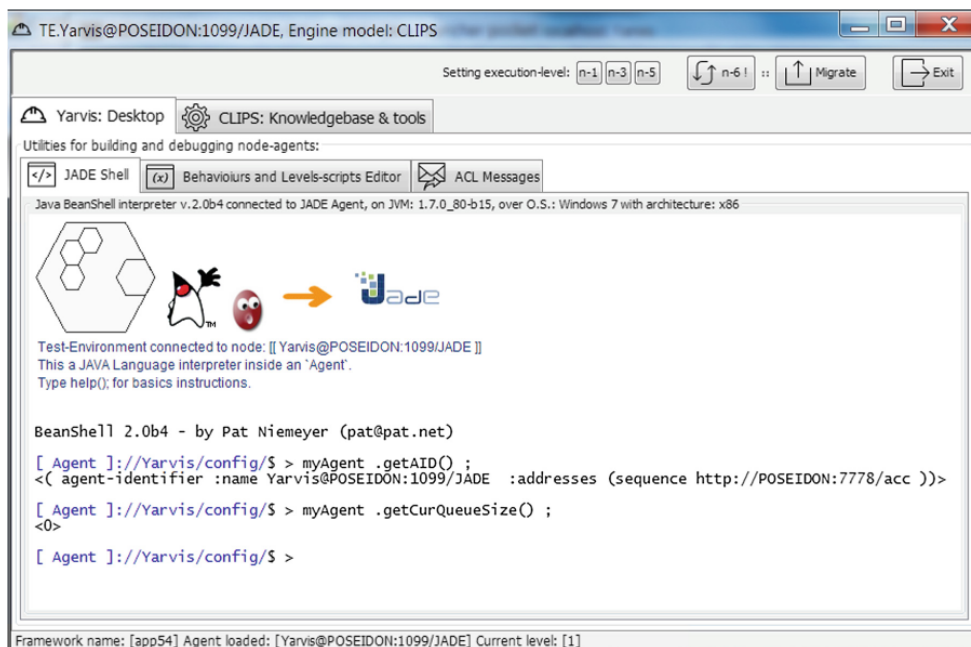
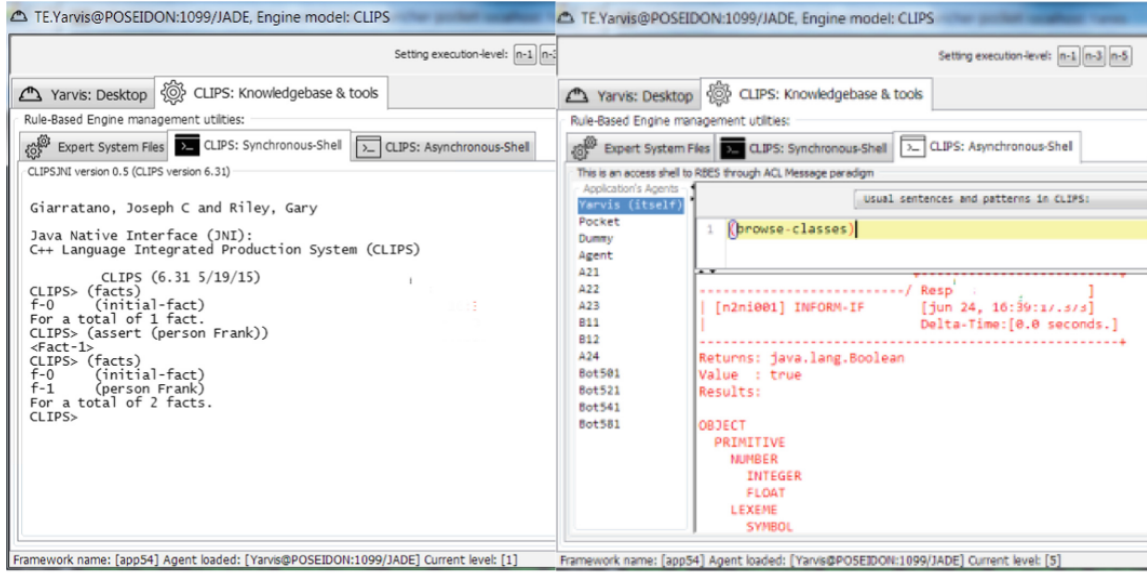


Fig. 2. BeanShell window in a rule-based agent.

A fully functional, self-contained test environment, along with some extra functionalities, is available for download and use at the following URL: <http://cekb.unileon.es/MAS/WORLDCIST19/>, instructions for deployment and use are included. This environment may be used for educational purposes, or as a multiagent-based development environment, as presented in [15].



(a)

(b)

Fig. 3. Synchronous (a) and asynchronous (b) GUI windows in a rule-based agent.

5 Discussion and Conclusion

This work describes a complete framework for the integration of a rule engine into an agent with the objective of building rule-based (or rule enhanced) agents able to communicate and cooperate in a multiagent platform.

The integration is made by using an interface that is used for interacting with an object representing the rule engine. The communication between rule-based agents is built by using FIPA compliant ACL messages transporting a set of constructs described in an ontology that represent a limited set of actions that can be performed in the rule engines of the agents.

The approach was inspired and is similar, to some extent, to the ones in [6, 7, 10]. But with several important differences that are highlighted next.

Communication mechanisms between the agent and the inference engine have been proposed in [7] using the *store* function from Jess to pass information between the agent and the engine by means of a HashMap structure, or by using data structures (*deftemplate* construct from Jess) for accommodating the fields of the received ACL (Agent Communication Language) messages. These solutions, while being viable, only consider the integration of a given inference engine (Jess in this case), and use native functions of the rule-engine for performing part of the processing. Also, in the opposite direction, when the rule engine has performed its execution cycle and the results are ready to be sent to the original requester, some approaches use the rule engine for building this response by using ad-hoc functions programmed for this purpose. This breaks the decoupling of the agent from its rule engine and makes it dependable on the technology used.

The approach presented in this paper uses a careful separation between the tasks of the agent and those of the rule engine. The listening behavior (see Sect. 3.4) is

responsible for processing the ACL message targeted at the rule engine and generate the corresponding calls to the RBE engine API. On the other hand, the response behavior is responsible for accessing the standard I/O of the rule engine to obtain whatever value is needed for building the response message.

The EMERALD framework [13] uses the “reasoning as a service” paradigm where the so-called Reasoner agent may ask any external inference engine that can perform the reasoning needed. This supposes a very loose coupling between the agent and the rule engine and, to some extent, breaks the encapsulation of the agent’s functionalities, rising an important concern about trust that must be addressed. The vision used in the present research is quite different in this sense. Though our approach also uses an uncoupled schema because of the use of an interface for accessing the rule engine, this rule engine is created in the moment of the incorporation of the agent into the platform, and not as part of the lifecycle of a behavior of the agent, as is the case in other approaches, and it is exclusively devoted to perform the reasoning activities of that agent in particular.

Apart from the careful decoupling between the actions that the rule engine must perform and those pertaining to the agent, which were discussed in the previous paragraphs, the solution proposed also introduces an original way of interacting between the agent and its rule engine. This interaction consists of the use of a FIPA compliant interaction protocol for this purpose. This interaction protocol-based implementation is used for communications not only with the own internal engine of a given agent but also with other engines in other remote agents in the platform. These interactions, then, adhere to the recommended structure for the dialog sequence among agents having *Rational Effect* (RE) as defined by FIPA under the denomination *FIPA-Request protocol*.

When an agent receives an ACL message, it first analyzes and decides if the requested actions are accepted. In the case they are accepted, the receiving agent auto-assigns again the received requests as a new ACL message aimed at itself (and notifies the sender about this acceptance). This allows to manage, internally and privately, the execution and the responses issued by the inference engine, and also makes room for implementing extra security, privacy or trust verifications in the agent before delivering the message to the rule engine. Though these issues have not been addressed in depth in this research, a simple verification of the sender agent was built in order to see if it is listed in an access list of trusted agents of the platform.

The combination of the integration and communication solutions between the rule engine and the agent allows a non-blocking execution of the agent activities during the message interchange or the inference tasks. Moreover, if new messages arrive at the receiving agent with new actions for the rule engine, these can be stored in the message queue once accepted by the agent, preserving the order in which they may arrive. This may be important for some interaction or coordination mechanisms among agents in a multiagent environment.

Some limitations of the approach, in its current implementation, include the lack of testing of the system by using the most usual agent coordination and collaboration schemas, only some basic ones have been tested. Another limitation is the need of further study for making the approach compatible with knowledge representation and reasoning technologies from the Semantic Web. In this sense, future work includes

implementing and studying these limitations, as well as building a proof-of-concept application in a practical and useful scenario. Also, security and trust issues should also be addressed and incorporated into the framework.

Acknowledgments. The work presented in this paper was supported by Junta de Castilla y León [grant number LE078G18].

References

1. Coulouris, G., Dollimore, J., Kindberg, T.: Distributed Systems: Concepts and Design, 5th edn. Cambridge University Press, New York (2012)
2. Durfee, E.H., Lesser, V.R., Corkill, D.D.: Trends in cooperative distributed problem solving. *IEEE Trans. Knowl. Data Eng.* **KDE-1**, 63–83 (1989)
3. Costin, B., Braubach, L., Paschke, A.: Rule-Based Distributed and Agent Systems (2011)
4. Bellifemine, F., Poggi, A., Rimassa, G.: JADE: a FIPA2000 compliant agent development environment. In: International Conference on Autonomous Agents and Multiagent Systems (2001)
5. Beneventi, A., Poggi, A., Tomaiuolo, M., Turci, P.: Integrating Rule and Agent-Based Programming to Realize Complex Systems. *Learning* (2004)
6. Poggi, A., Tomaiuolo, M.: Rule engines and agent-based systems. In: Encyclopedia of Artificial Intelligence, pp. 1404–1410. IGI Global (2009)
7. Cardoso, H.L.: Integrating JADE and Jess. http://jade.tilab.com/doc/tutorials/jade-jess/jade_jess.html
8. Friedman-Hill, E.J., et al.: Jess the Java Expert System Shell. *Distrib. Comput. Syst.* Sandia Natl. Lab., USA (1997)
9. Poggi, A., Tomaiuolo, M., Turci, P.: An agent-based service oriented architecture. In: WOA 2007, Genova, pp. 157–165 (2007)
10. Christoph, U., Krempels, K.-H., Wilden, A.: JAMOGAAGENT a rule-based programmable agent. In: ICAART 2009 Proceedings of the International Conference on Agents and Artificial Intelligence, pp. 447–454 (2009)
11. Giarratano, J.C.: CLIPS 6.4 user's guide (2014)
12. Niemeyer, P.: Beanshell - The Lightweight scripting for Java
13. Bassiliades, N.: Agents and knowledge interoperability in the semantic web era. In: Proceedings of the 2nd International Conference on Web Intelligence, Mining and Semantics- WIMS 2012, vol. 1 (2012)
14. Paschke, A., Boley, H.: Rule responder: rule-based agents for the semantic-pragmatic web. *Int. J. Artif. Intell. Tools* **20**, 1043–1081 (2011)
15. Aguayo, F.J., García, I., Alaiz-Moretón, H., Benavides, C.: Techniques and utilities to improve the design, development and debugging of multiagent applications with agile principles. In: Advances in Intelligent Systems and Computing (2018)